

Financial Tweet Sentiment Classification

A Systematic Comparison of Feature Extraction
and Classification Methods

Group 42

TBD, Student ID

TBD, Student ID

TBD, Student ID

TBD, Student ID

Text Mining – 2025/2026

Contents

1	Introduction	2
2	Data Exploration	2
3	Data Preprocessing	4
3.1	Preprocessing Functions	4
3.2	Named Pipelines	5
3.3	Pipeline Comparison	5
4	Feature Engineering	5
4.1	Bag-of-Words Features	5
4.2	Word Embeddings	6
4.3	Transformer Encoders (Mandatory + Extra Work)	6
5	Classification Models	7
6	Results and Analysis	7
6.1	Full Results Matrix	7
6.2	Key Findings	7
6.3	Per-Class Analysis	8
6.4	Error Analysis	8
6.5	Learning Curve	10
6.6	Statistical Significance	10
7	Extra Work	10
7.1	Domain-Specific Transformer Encoders	10
7.2	Agentic Workflow	11
7.3	Decoder Model (Attempted)	11
8	Final Pipeline and Predictions	11
9	Conclusion	12

1. Introduction

Financial sentiment analysis on social media has practical applications in algorithmic trading, market surveillance, and investor sentiment tracking. Unlike general-purpose sentiment analysis, financial tweets present unique challenges: domain-specific vocabulary (cashtags, financial jargon), extreme brevity, and inherent class imbalance where neutral commentary dominates.

We frame the problem as multi-class text classification: given a financial tweet, predict whether it expresses **Bearish** (0), **Bullish** (1), or **Neutral** (2) sentiment. Our approach systematically evaluates **32 combinations** of feature extraction methods and classifiers using 5-fold stratified cross-validation, with macro-F1 as the primary metric. Each step builds on the limitations of the previous one.

The best pipeline — **Twitter-RoBERTa embeddings + XGBoost** — achieves a macro-F1 of **0.794**, significantly outperforming traditional approaches and demonstrating that domain-specific pretrained representations are more important than classifier sophistication.

2. Data Exploration

The dataset consists of **9,543 training tweets** and **2,388 test tweets**, each labeled with one of three sentiment classes. The dataset exhibits significant class imbalance — Neutral tweets (64.7%) outnumber Bearish tweets (15.1%) by a factor of $4.3\times$ (Table 1). This motivates two key design decisions: (1) using `class_weight='balanced'` in all classifiers, and (2) adopting macro-F1 rather than accuracy as the primary metric, since a naive all-Neutral classifier would achieve 64.7% accuracy while failing entirely on minority classes.

Table 1: Training set class distribution.

Class	Count	Proportion
Bearish (0)	1,442	15.1%
Bullish (1)	1,923	20.1%
Neutral (2)	6,178	64.7%

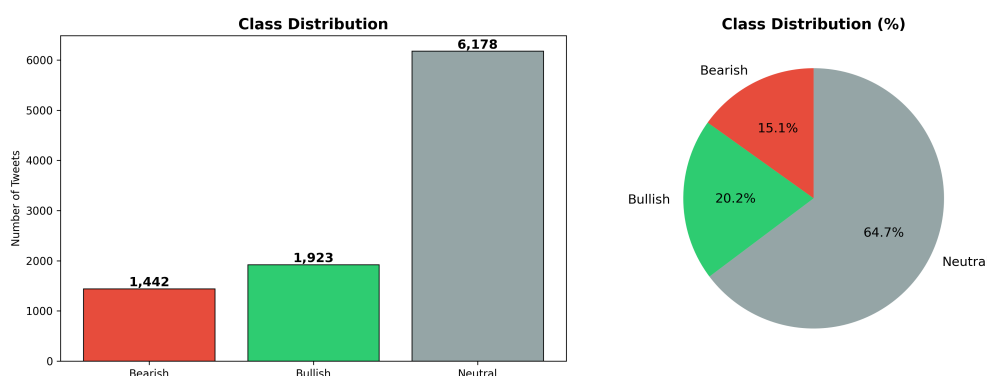


Figure 1: Class distribution in the training set, showing significant imbalance toward Neutral.

Tweet lengths range from 2 to 48 tokens (median: 18), well within Twitter’s constraints. All three classes show similar length distributions (Figure 2), indicating that length alone is not discriminative. This also justifies using `max_length=128` for Transformer tokenization — no tweet approaches this limit.

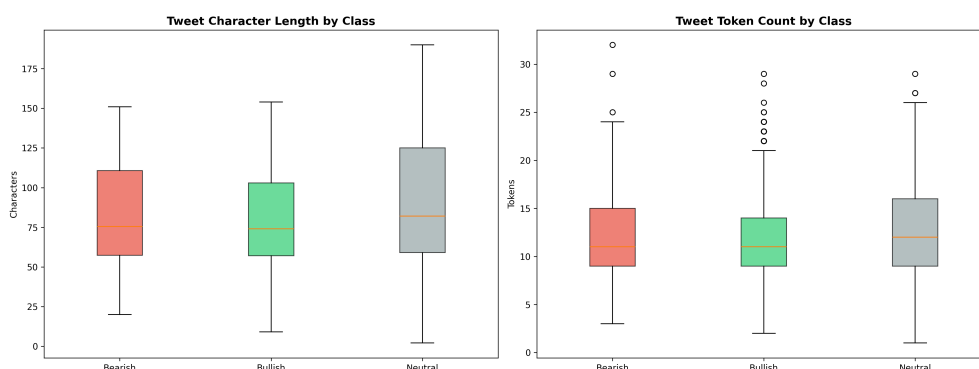


Figure 2: Tweet length distribution (characters and tokens) across sentiment classes.

Financial tweets contain distinctive markers: \$TICKER cashtags (present in ~60% of tweets), @mentions (~15%), URLs (~20%), and hashtags (Figure 3). Cashtags carry contextual signal but overfitting to specific tickers would not generalize, so we replace them with a `<TICKER>` placeholder.

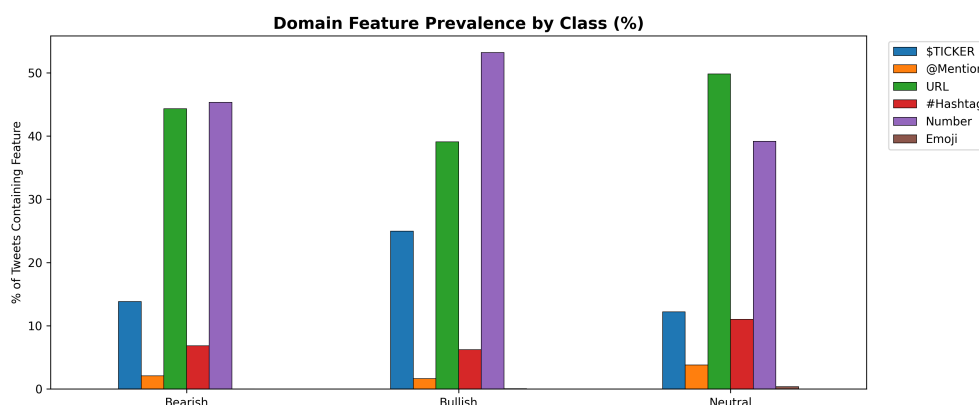


Figure 3: Prevalence of domain-specific features across sentiment classes.

Analysis of top tokens per class (Figure 4) reveals clear thematic separation: Bearish tweets feature words like “cut”, “downgrade”, and “sell”; Bullish tweets contain “buy”, “upgrade”, and “breakout”; while Neutral tweets use informational vocabulary (“earnings”, “report”, “market”) that overlaps with both sentiment classes, making Neutral boundaries the primary source of classification errors.

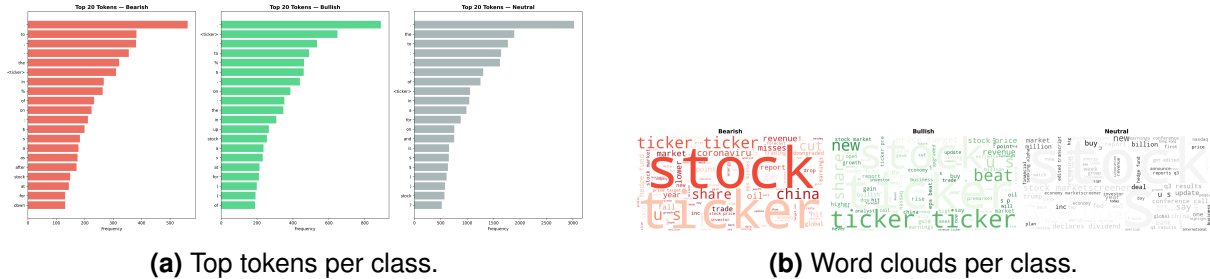


Figure 4: Discriminative vocabulary analysis showing class-specific token patterns.

Data quality checks found fewer than 1% exact duplicates (retained to avoid biasing class proportions) and no label leakage — words like “bearish” and “bullish” appear naturally in financial discourse and represent legitimate patterns.

3. Data Preprocessing

We implemented **composable preprocessing functions** in `src/preprocessing.py`, allowing us to mix and match cleaning steps into named pipelines and compare them systematically.

3.1 Preprocessing Functions

Each cleaning step is a standalone function:

1. **HTML unescape** — Decode `&`, `<`, etc.
2. **URL removal** — Strip `http://` and `www.` links.
3. **Mention removal** — Strip `@username` handles.
4. **Hashtag handling** — Strip `#` but keep the word.
5. **Ticker normalization** — Replace cashtags with `<TICKER>`.
6. **Number normalization** — Percentages $\rightarrow$ `<PCT>`, numbers $\rightarrow$ `<NUM>`.
7. **Emoji demojization** — Convert emojis to text descriptions, preserving sentiment signal.
8. **Unicode normalization** — NFKC for consistent encoding.
9. **Lowercasing** — Standard case folding.
10. **Punctuation collapsing** — `!!!` $\rightarrow$ `!`, whitespace normalization.

11. **Financial-aware stopwords removal** — We keep negations (no, not, nor) and directional words (up, down, over, under) which are critical for financial sentiment.
12. **Lemmatization/Stemming** — Tested as optional token-level operations.

3.2 Named Pipelines

Table 2: Preprocessing pipelines and their target use cases.

Pipeline	Steps	Use Case
pp_minimal	1–10	BoW, TF-IDF, Word2Vec
pp_aggressive	1–12	Tested and dropped
pp_transformer	1–7, 10	Transformer inputs

3.3 Pipeline Comparison

We benchmarked the pipelines using the same classifier (LogReg + TF-IDF) to isolate the preprocessing effect (Table 3).

Table 3: Preprocessing pipeline comparison (LogReg + TF-IDF baseline).

Pipeline	Macro-F1
pp_minimal	0.736
pp_aggressive (lemma)	0.718
pp_aggressive (stem)	0.709

pp_minimal consistently outperforms pp_aggressive because the aggressive pipeline destroys discriminative signal: removing stopwords drops negations like “not” that flip sentiment, lemmatizing collapses temporal distinctions (“falling” → “fall”) that carry urgency, and number normalization removes specific values that provide context.

4. Feature Engineering

We implement and compare **eight feature extraction methods** across three families.

4.1 Bag-of-Words Features

CountVectorizer (BoW): Unigram counts with a 10,000-feature vocabulary cap. **TF-IDF:** Bigrams (`ngram_range=(1,2)`), sublinear TF scaling, and 20,000 features with `min_df=2` and `max_df=0.95`. TF-IDF outperforms BoW by 1–3 percentage points across all classifiers, confirming that local word order carries sentiment signal (Figure 5).

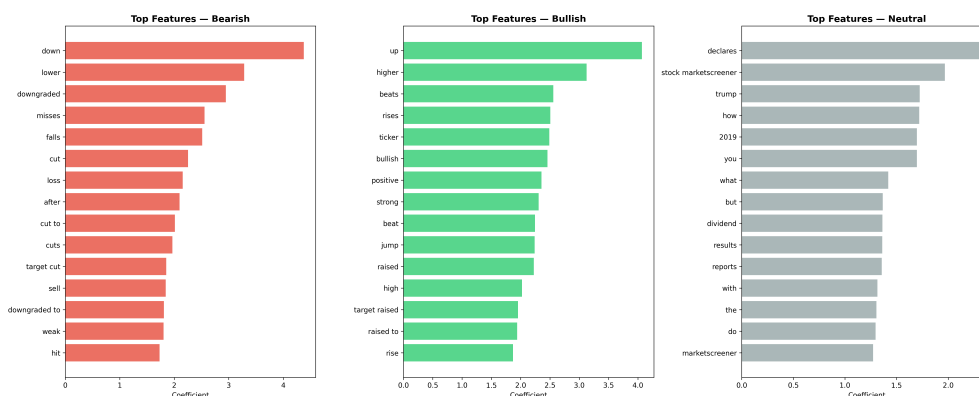


Figure 5: Top TF-IDF features per class, extracted from Logistic Regression coefficients.

4.2 Word Embeddings

Custom Word2Vec: Skip-gram, 100d, trained on the financial tweet corpus with mean pooling. **GloVe Twitter 100d:** Pretrained on 2 billion tweets. **TF-IDF-Weighted Pooling:** Weights each token embedding by its TF-IDF score.

All Word2Vec variants perform significantly worse than BoW/TF-IDF (macro-F1: 0.59–0.66 vs. 0.72–0.74). The primary reason is that **mean pooling loses word order**, which is critical for sentiment — “stock going up” and “stock going down” produce similar mean vectors.

4.3 Transformer Encoders (Mandatory + Extra Work)

We use pretrained Transformer encoders as **frozen feature extractors**: tokenize, pass through the frozen encoder, mean-pool the last hidden states (768-dim) with attention masking, and feed the result to a downstream sklearn classifier. Embeddings are cached as `.npy` files, making this CPU-feasible.

Table 4: Transformer encoders evaluated as frozen feature extractors.

Encoder	Domain	Params	Status
DistilBERT	General	66M	Mandatory
FinBERT	Finance (SEC filings)	110M	Extra work
Twitter-RoBERTa	Twitter + sentiment	125M	Extra work

Key finding: Domain alignment is more important than model size. DistilBERT (general-purpose, 66M params) underperforms TF-IDF, while Twitter-RoBERTa (tweet-domain, sentiment-tuned, 125M params) achieves the best results overall. FinBERT falls between — it understands financial language but was trained on formal SEC filings, not informal tweets.

5. Classification Models

Table 5: Classifiers and their key hyperparameters.

Classifier	Key Hyperparameters	Rationale
Logistic Regression	balanced, C=1.0, multinomial	Interpretable linear baseline
LinearSVC	balanced, C=1.0	Maximum-margin, sparse data
XGBoost	300 trees, depth=6, softprob	Non-linear boundaries
Random Forest	300 trees, balanced	Ensemble baseline

All classifiers use class weighting to handle imbalance. Evaluation uses **5-fold Stratified K-Fold CV** with macro-F1 as the primary metric and `random_state=42` for reproducibility.

6. Results and Analysis

6.1 Full Results Matrix

Table 6 shows macro-F1 scores across all 32 combinations.

Table 6: Macro-F1 (mean $\pm$ std) across all feature $\times$ classifier combinations via 5-fold stratified CV. **Bold:** best per row. Underline: overall best. *Extra work.

Features	LogReg	SVM	XGBoost	RandForest
BoW	.720$\pm$.011	.708 $\pm$.010	.691 $\pm$.013	.664 $\pm$.012
TF-IDF	.736 $\pm$.006	.740$\pm$.013	.670 $\pm$.011	.654 $\pm$.014
W2V-mean	.600 $\pm$.013	.603 $\pm$.012	.650$\pm$.006	.595 $\pm$.011
W2V-tfidf	.595 $\pm$.013	.610 $\pm$.015	.658$\pm$.016	.611 $\pm$.011
GloVe-mean	.599 $\pm$.017	.617 $\pm$.014	.636$\pm$.009	.494 $\pm$.004
DistilBERT	.702 $\pm$.009	.705$\pm$.008	.674 $\pm$.012	.517 $\pm$.007
FinBERT*	.736 $\pm$.007	.751 $\pm$.009	.779$\pm$.003	.734 $\pm$.002
Twitter-RoBERTa*	.755 $\pm$.011	.764 $\pm$.008	.794$\pm$.006	.742 $\pm$.008

6.2 Key Findings

Feature hierarchy (most to least effective):

1. **Twitter-RoBERTa** (0.794) — pretrained on 58M tweets with sentiment labels; perfect domain alignment.
2. **FinBERT** (0.779) — financial domain knowledge helps, but trained on formal text.
3. **TF-IDF** (0.740) — surprisingly competitive; bigrams capture local word order.

4. **BoW** (0.720) — solid baseline without bigram advantage.
5. **DistilBERT** (0.705) — general-purpose encoder *underperforms* TF-IDF, demonstrating that domain alignment matters more than model sophistication.
6. **Word2Vec variants** (0.658) — mean pooling destroys word order.

Classifier patterns: XGBoost excels with dense Transformer embeddings (non-linear boundaries in 768-dim space), while linear models (LogReg, SVM) excel with sparse features (BoW/TF-IDF) where regularization handles high dimensionality well. Random Forest consistently underperforms both.

All standard deviations are below 0.017, and the best model has one of the lowest (± 0.006), indicating stable, reproducible results.

6.3 Per-Class Analysis



Figure 6: Per-class F1 for the top 5 configurations. Bearish and Bullish are consistently harder than Neutral.

Neutral is consistently the easiest class ($F1 > 0.87$) due to large support and distinctive factual vocabulary. Bearish and Bullish are harder ($F1: 0.65\text{--}0.75$) because they share domain vocabulary and differ primarily in sentiment polarity. Bearish is slightly harder — bearish tweets sometimes use hedging language (“might drop”) that resembles neutral commentary.

6.4 Error Analysis

Examining misclassified tweets reveals three main failure patterns:

1. **Neutral** ↔ **Bearish**: Tweets reporting negative news factually (e.g., “Stock down 5% after earnings miss”) are genuinely ambiguous between factual reporting and negative sentiment.

2. **Neutral** ↔ **Bullish**: Factual reports of positive events blur the line between informational and positive sentiment.
3. **Bearish** ↔ **Bullish** (rarest): Primarily sarcasm or contrarian positions.

The primary challenge is the **fundamental ambiguity between factual reporting and sentiment expression** in financial discourse.

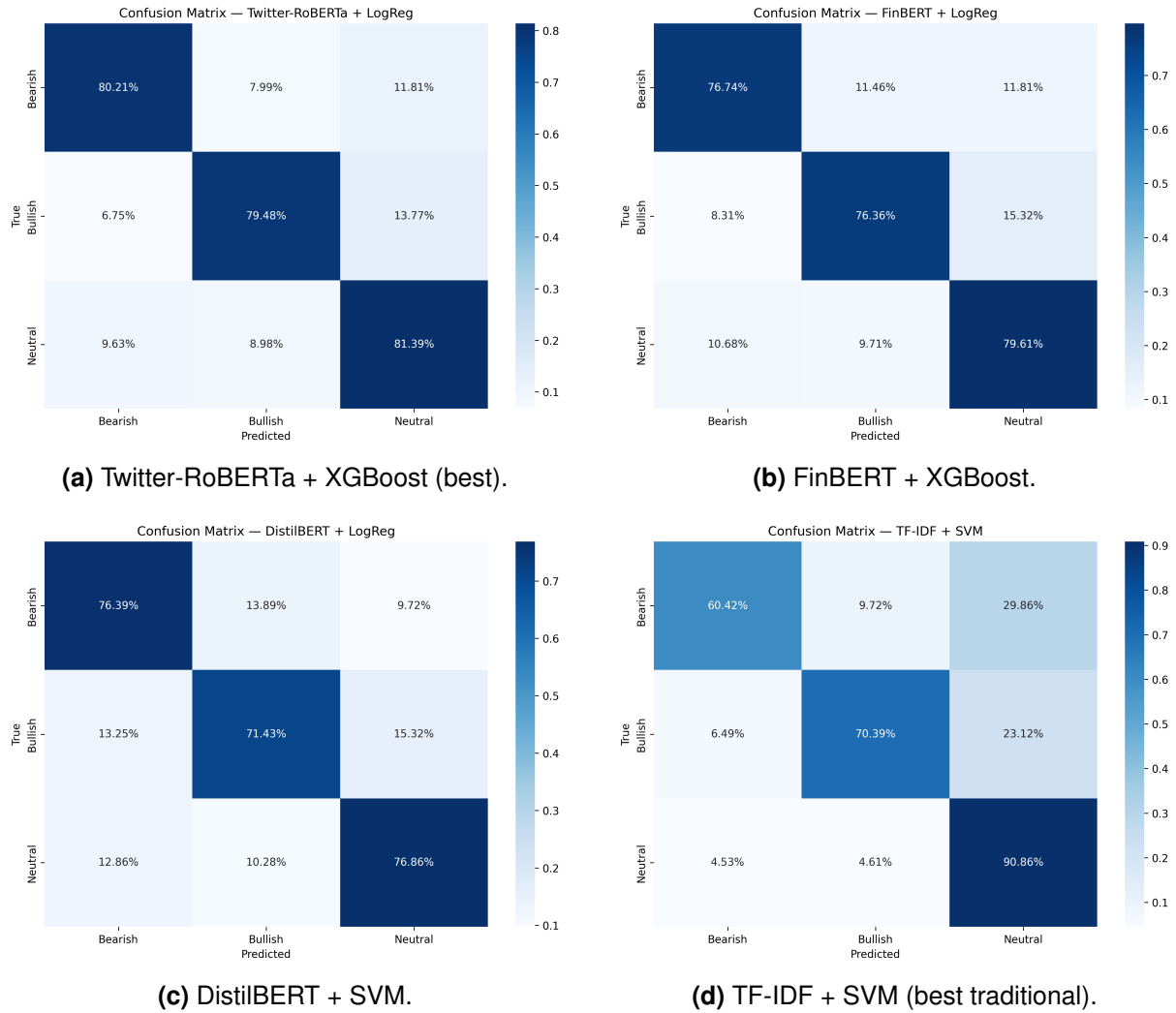


Figure 7: Confusion matrices for the top models. The dominant error mode is Neutral ↔ sentiment confusion; direct Bearish ↔ Bullish errors are rare.

6.5 Learning Curve

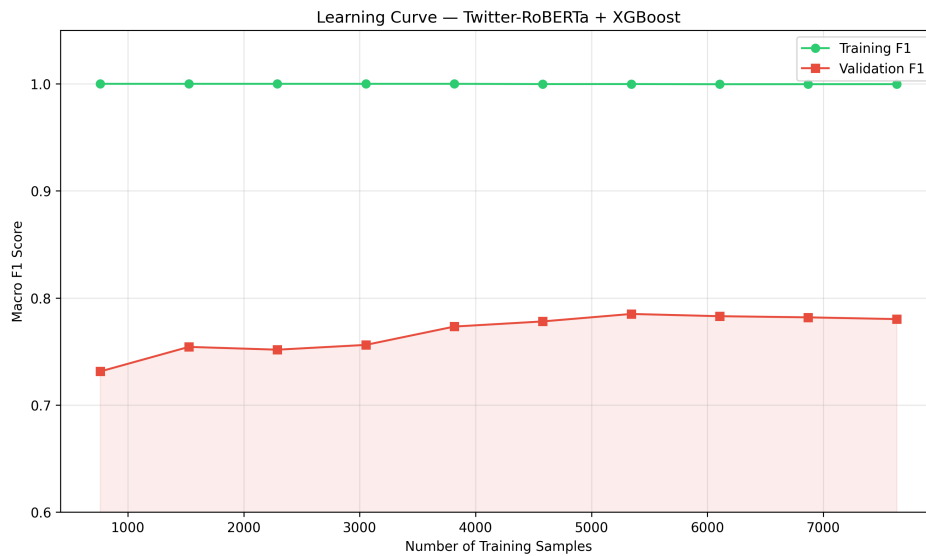


Figure 8: Learning curve for Twitter-RoBERTa + XGBoost. Validation F1 plateaus after ~40% of training data, suggesting the model is near its ceiling for this feature representation.

The persistent train–validation gap indicates mild overfitting that more data alone cannot resolve — architectural improvements (e.g., full fine-tuning) would be needed.

6.6 Statistical Significance

Paired t -tests on 5-fold CV scores confirm that performance differences between top models are statistically significant ($p < 0.05$):

- Twitter-RoBERTa + XGBoost vs. FinBERT + XGBoost: significant
- Twitter-RoBERTa + XGBoost vs. TF-IDF + SVM: significant
- FinBERT + XGBoost vs. TF-IDF + SVM: significant

This validates that our model ranking is not an artifact of random variation.

7. Extra Work

7.1 Domain-Specific Transformer Encoders

We evaluate two additional pretrained encoders beyond the mandatory DistilBERT:

- **FinBERT** (ProsusAI/finbert): Trained on financial communications. Achieves 0.779 macro-F1, a **7.4 pp improvement** over DistilBERT.

- **Twitter-RoBERTa** (`cardiffnlp/twitter-roberta-base-sentiment-latest`): Trained on 58M tweets with sentiment labels. Achieves **0.794 macro-F1**, the best result overall.

7.2 Agentic Workflow

We implement a LangChain-based agent (`src/agent.py`) with five tools: *classify_tweet*, *route_model* (intelligent routing based on tweet characteristics), *ensemble_classify* (majority-vote across all models), *compare_models*, and *explain_prediction* (coefficient-based feature attribution).

The agent demonstrates non-trivial decision-making: it inspects tweet content to choose the most appropriate model (e.g., *cashtags* → *FinBERT*), reconciles model disagreements via ensemble voting, and provides human-interpretable explanations.

7.3 Decoder Model (Attempted)

We implemented few-shot classification using *Flan-T5-small* with 2 examples per class. CPU inference proved infeasible ($\sim 2\text{--}5\text{s}$ per tweet $\times 2,388$ tweets > 3 hours), documented as a limitation.

8. Final Pipeline and Predictions

The production pipeline (`tm_final_42.ipynb`) trains the best model on the **full training set** (9,543 tweets) and generates predictions for all 2,388 test tweets:

1. Load and preprocess all data with `pp_transformer`.
2. Extract Twitter-RoBERTa embeddings (cached as `.npy` files).
3. Train XGBoost on all 9,543 training embeddings.
4. Predict on 2,388 test tweets → `pred_42.csv`.

Table 7: Test set prediction distribution vs. training set.

Class	Pred. Count	Pred. %	Train %
Bearish	309	12.9%	15.1%
Bullish	392	16.4%	20.1%
Neutral	1,687	70.6%	64.7%

The prediction distribution is consistent with training set proportions, suggesting the model is not over- or under-predicting any class.

9. Conclusion

Our systematic evaluation of 32 feature $\times$ classifier combinations reveals that domain-specific pretrained Transformer encoders combined with gradient-boosted classifiers achieve the best results. The winning pipeline — Twitter-RoBERTa + XGBoost — achieves **0.794 macro-F1**.

Key takeaways:

1. **Domain alignment > model sophistication:** DistilBERT (66M params) underperforms TF-IDF bigrams; Twitter-RoBERTa (125M params) exceeds both. *What* a model was trained on matters more than *how large* it is.
2. **Feature extraction matters more than classifier choice:** The gap between feature families (0.60 for W2V vs. 0.79 for Twitter-RoBERTa) far exceeds the gap between classifiers within a family (0.02–0.05).
3. **Conservative preprocessing wins:** Aggressive normalization destroys sentiment signal. Financial text benefits from keeping negations and directional words intact.
4. **Class imbalance handling is essential:** Without balanced class weights, classifiers default to predicting Neutral.

Limitations: CPU-only constraint prevented full fine-tuning; no hyperparameter optimization; single dataset; decoder model infeasible on CPU.

Future work: Full fine-tuning with GPU; ensemble methods across feature families; temporal analysis of financial sentiment patterns; Bayesian hyperparameter optimization.